

Porting `igsca_tree` / `igsca_ctree` into `doTrees()`

What is missing, what is wrong, and what to change — branch `igscaTrees`

1 Scope and method

This reviews the port of `dev/igsca/tree_template/igsca_tree.R` and `igsca_testers.R` into `R/postestimate_doTrees.R` + `R/helper_doTrees.R`, and the new `tests/testthat/test-postestimate_doTrees.R`.

Every finding below was **executed**, not read off. All five `influence` values were run against `tests/testthat/data/igscaTrees.Rdata` (`n = 1000`, covariates `z_true` (factor), `noise_1`, `noise_2`) under `devtools::load_all()`, with reduced `R_test/maxdepth` where the default control was too slow to finish. Verbatim error text is quoted.

Headline: the COIN branch (`influence = "mat" / "vec"`, `splitter = NULL`) works end to end. Everything else is broken — the three partition-family methods cannot run at all, and two of the three mixed-splitter configurations fail **silently** in a way the current tests are structurally unable to detect.

2 Blockers — the partition family cannot run

2.1 B1. `idx_permutation()` and `ndt_dists()` were never ported

Location: R/helper_doTrees.R:699, R/helper_doTrees.R:635

The comment at R/helper_doTrees.R:535 still says these come from R/MGA/csem_test_helpers.R. That file was not ported, and neither function exists anywhere in the package:

```
$ grep -rn "idx_permutation\|ndt_dists" R/
R/helper_doTrees.R:535:# R/MGA/csem_test_helpers.R (try_fit, idx_permutation,
ndt_dists).
R/helper_doTrees.R:581: #' (NPT) or an ndt_dists() distance name -- "DGi"/"DLi"
only: ...
R/helper_doTrees.R:635:      ndt_dists(coll$ndt_pools$Sc, coll$ndt_pools$Si, mga$fit)
R/helper_doTrees.R:699: perm <- idx_permutation(n, R)
```

Only comments and call sites — no definitions. Confirmed at runtime:

```
doTrees(dat, model, covs, influence = "FIT", splitter = "FIT",
        control = igsca_tree_control(R_test = 2L))
#> Error: could not find function "idx_permutation"
```

`idx_permutation()` gates **all three** partition selectors (it is on the unconditional path in `permutation_pvalue()`), and `ndt_dists()` additionally gates `DGi/DLi` everywhere, including as `igsca_ctree`-style mixed splitters.

Fix. Port both. The contracts implied by the call sites are:

```
#' @return An `R` x `n` integer matrix; row r is one permutation of seq_len(n).
idx_permutation <- function(n, R) t(replicate(R, sample.int(n)))

#' @return Named numeric: "DGc", "DLc", "DGi", "DLi".
ndt_dists <- function(Sc, Si, mga_fit) {
  Sc2 <- bdiagFit(mga_fit, .type_vcv = "construct")
  Si2 <- bdiagFit(mga_fit, .type_vcv = "indicator")
  c(DGc = calculateDG(.matrix1 = Sc, .matrix2 = Sc2),
    DLc = calculateDL(.matrix1 = Sc, .matrix2 = Sc2),
    DGi = calculateDG(.matrix1 = Si, .matrix2 = Si2),
    DLi = calculateDL(.matrix1 = Si, .matrix2 = Si2))
}
```

The $R \times n$ orientation is not optional — `permutation_pvalue()` indexes `goes_left[perm[r, ]]`, so a transposed return silently produces a recycled-length nonsense null distribution rather than an error.

I verified the rest of the partition path with exactly these two definitions injected: `grow_extree()`, `partition_select()`, the scan cache, `party()` construction, `print`, and `predict(type = "node")` all behave. Root criteria for `influence = "FIT"` came back sensible, including the deliberate `.Machine$double.eps` floor on a negative statistic:

	z_true	noise_1	noise_2
statistic	0.04315312	0.001322932	2.220446e-16
p.value	0.25000000	0.25000000	5.000000e-01
criterion	-0.28761311	-0.287647590	-6.931472e-01

So B1 is the only thing standing between the current code and a working partition family. Everything downstream of it is sound.

2.2 B2. splitter means two incompatible things in the two branches

Location: R/postestimate_doTrees.R:106--133 vs R/postestimate_doTrees.R:154--163

In the ctree branch `splitter` must be a **function** (it is handed straight to `argmax_split()`). In the partition branch it must be a **string** — it is validated with `%in%` and then `switch()`ed:

```
stopifnot(
  "splitter should be one of 'FIT', 'DLi' or 'DGi'" = splitter %in%
  c("FIT", "DLi", "DGi")
)
splitter <- switch(splitter, FIT = split_max_fitdiff, ...)
```

The tests pass **functions** to both branches, so every partition-family test errors. There are two distinct failure modes:

```
# tests/testthat/test-postestimate_doTrees.R:117 -- splitter is a function
doTrees(dat, model, covs, influence = "FIT", splitter = split_max_fitdiff, ...)
#> Error: 'match' requires vector arguments

# tests/testthat/test-postestimate_doTrees.R:154 -- splitter omitted (NULL)
doTrees(dat, model, covs, influence = "DLi", ...)
#> Error: EXPR must be a length 1 vector
```

The second is the nastier one and deserves a note of its own: `NULL %in%` `c("FIT", "DLi", "DGi")` is `logical(0)`, and `stopifnot()` accepts `logical(0)` vacuously (`all(logical(0))` is `TRUE`). So the guard that exists specifically to catch this input **passes it through**, and the failure surfaces one line later as an opaque `switch()` error naming neither the argument nor the function. Any `stopifnot(... = x %in% y)` in this package has the same hole whenever `x` can be `NULL`.

Fix. Make `splitter` a character string on both paths, with an explicit “use the engine’s native scan” level, and resolve it once before branching:

```
doTrees <- function(data, model, covariates,
  influence = c("mat", "vec", "FIT", "DLi", "DGi"),
  splitter = c("native", "FIT", "DLi", "DGi"),
  control = igsca_tree_control()) {
  influence <- match.arg(influence)
  splitter <- match.arg(splitter)

  split_fn <- switch(splitter,
    native = NULL,
    FIT = split_max_fitdiff,
    DLi = split_max_dli,
    DGi = split_max_dgi
  )
  ...
}
```

`match.arg()` replaces the broken `stopifnot() + switch()` pair, gives a readable error for a bad value, and makes “native” a legal, documented, self-describing default instead of a bare `NULL` that means different things in different branches. Note this changes the tests’ call signature from `splitter = split_max_dli` to `splitter = "DLi"`.

3 Silent failures

3.1 S1. A missing splitter statistic degrades to a stump with no error

Location: R/helper_doTrees.R:310--319

`argmax_split()` wraps each kernel call in `tryCatch(..., error = function(e) NA_real_)`. That is correct for a genuinely non-identified candidate partition, but it also swallows **programming** errors — including B1's could not find function "ndt_dists". Every candidate becomes NA, `any(is.finite(stats))` is FALSE, the loop nexts through every covariate, and `argmax_split()` returns NULL, which partykit reads as “no admissible split”.

Measured, on today's code:

```
doTrees(dat, model, covs, influence = "mat", splitter = split_max_dli,
        control = igsca_tree_control(R_test = 50L, maxdepth = 1L))
#> Fitted party:
#> [1] root: *
#> Number of inner nodes:    0
#> Number of terminal nodes: 1
```

No warning, no error, `n_fail_resample == 0`. The same call with `splitter = split_max_fitdiff` (whose kernel **does** resolve) splits on `z_true` as expected. So the DLi/DGi mixed splitters are currently dead code that returns a plausible-looking object, and `expect_no_error()` certifies them as passing.

Fix. Distinguish “this candidate is not estimable” from “this kernel is broken”. Cheapest version — validate the kernel once, outside the scan:

```
stats <- vapply(cands, function(cc) {
  tryCatch(splitter(model, mf, subset, cc$goes_left, ctrl),
    error = function(e) NA_real_)
}, numeric(1))
if (!any(is.finite(stats))) {
  collector$n_fail_split <- collector$n_fail_split + 1L
  next
}
```

and surface a `warning()` from `doTrees()` when a non-“native” splitter was requested but `n_fail_split` equals the number of nodes visited. Recording the count is the part that matters: right now a totally dead kernel and a genuinely unsplittable node are indistinguishable in the returned object.

3.2 S2. `control$bonferroni` is silently ignored on the `ctree` path

Location: R/postestimate_doTrees.R:104--105

The template had one more line after `ctree_control()`:

```
cc <- partykit::ctree_control(...)
cc$bonferroni <- isTRUE(control$bonferroni) # <- dropped in the port
```

It is gone, replaced by `# TODO: Return to this bonferroni problem`. This is not a no-op. `ctree_control()` derives `bonferroni` from `testtype`, but the port computes `testtype` from `coin_distribution` first:

```
ctree_control()$bonferroni TRUE (testtype = "Bonferroni")
ctree_control(testtype = "MonteCarlo")$bonferroni FALSE
```

With the default `coin_distribution = "approximate"` the port always sends `testtype = "MonteCarlo"`, so `cc$bonferroni` is always `FALSE` regardless of what the user asked for. `igsca_tree_control(bonferroni = TRUE)` is accepted, stored, threaded through, and then discarded.

The asymmetry is the real problem: on the **partition** path `grow_extree()` copies the whole control list into `ctrl` (R/helper_doTrees.R:249--251), so `bonferroni` **does** take effect there. One argument, honoured in one branch and ignored in the other, with no diagnostic.

Fix. Restore the line. To answer the TODO on its own terms: `testtype` selects how p-values are **computed** (permutation vs asymptotic), whereas `.extree_node()` reads `ctrl$bonferroni` independently to decide whether to **multiplicity-adjust** the criterion across covariates. They are orthogonal, which is exactly why the explicit assignment is needed — otherwise choosing `coin_distribution = "approximate"` covertly forces `bonferroni = FALSE`.

3.3 S3. `coef()` and `plot()` labels are empty for any tree that splits

Location: R/helper_doTrees.R:461--477 (`coef.igsca_tree`), R/helper_doTrees.R:490--507 (`plot.igsca_tree`)

Both methods read `nobs / objfun` out of `info_node()` at **terminal** nodes. `partykit` only calls the trafo at nodes it attempts to split, so leaves carry no info at all:

```
r <- doTrees(dat, model, covs, influence = "mat",
             control = igsca_tree_control(R_test = 50L, maxdepth = 1L))
# node 1 -> info names: criterion,p.value,converged,objfun,object,nobs
# node 2 -> info names: NULL
# node 3 -> info names: NULL

coef(r)
#> NULL
```

`coef()` returns `NULL` for every tree with at least one split (it only appeared to work in my stumps, where the root **is** the leaf). `plot()` does not error — `sprintf("n = %s", NULL)` yields `character(0)` — it just draws unlabelled terminal boxes, which is worse than failing.

This is a functional gap, not just a method bug: **the returned tree contains no per-leaf IGSCA fit**, which is presumably the main thing a user wants from a SEM tree. The leaf models are never estimated.

Fix. Fit the leaves once after growing, then have both methods read from there:

```
refit_leaves <- function(tree, mf, model, indicators) {
  ids <- partykit::nodeids(tree, terminal = TRUE)
  leaf <- partykit::fitted_node(partykit::node_party(tree), mf)
  stats::setNames(lapply(ids, function(id) {
    ft <- try_fit(mf[leaf == id, indicators, drop = FALSE], model)
    list(nobs = sum(leaf == id), converged = ft$ok, object = if (ft$ok) ft$fit)
  }), ids)
}
```

Store the result in `attr(ret, "igsca_info")$leaves` and rewrite `coef.igsca_tree()` / the default `plot.igsca_tree()` FUN against it. Cost is one IGSCA fit per leaf (~ 0.6 s each here), which is negligible next to growing the tree.

While there: `object = ft$fit` is stored in **every inner node's** info (R/
`postestimate_doTrees.R:79, :203`). Each is a full `cSEMResults` object, so a `maxdepth = 3` tree retains
up to 7 of them. Consider dropping `object` from the stored info once the leaf fits are attached
explicitly.

4 Test-suite findings

The test file is the right shape — one file per R file, five blocks covering the five methods, matching the conventions in `MEMORY.md`. These are the things that stop it from actually testing anything.

4.1 T1. Snapshots are stale and will fail

`tests/testthat/_snaps/postestimate_doTrees.md` records the printed object under the name `trees_out`:

```
# IGSCA Trees Conditional Test on Matrix of Residuals Runs as expected
```

```
Code
trees_out
```

The tests now bind `trees_mx` / `trees_vec` / `trees_NPT_FIT` / `expect_snapshot()` captures the **expression** as well as the output, so both existing blocks fail on the `Code` line alone, and the three partition blocks have no snapshot at all. Delete the file and regenerate once the code runs.

4.2 T2. `length(tree) == 5` is a disguised structural assertion

Location: `tests/testthat/test-postestimate_doTrees.R:32, :78, :125, :160, :194`

```
# Dirty substitute for snapshot
expect_true(length(trees_mx) == 5)
```

`length()` on a `party` dispatches to `length.party()`, which returns the **number of nodes**, not the length of a list. Measured: the same call with `maxdepth = 1L` gives `length(r) == 3`. So this is asserting “the tree has exactly 5 nodes” — a strictly weaker duplicate of the snapshot immediately below it, which will break on any legitimate change to the control defaults or the RNG stream, for reasons the comment actively misleads you about.

Drop it, or replace it with an assertion that says what it means:

```
expect_s3_class(trees_mx, "igsca_tree")
expect_identical(attr(trees_mx, "igsca_info")$n_fail_full, 0L)
```

4.3 T3. `expect_no_error()` cannot see the failure mode that actually occurs

Location: `tests/testthat/test-postestimate_doTrees.R:35--62` and the four analogous blocks

Three `doTrees()` calls are wrapped in a single `expect_no_error({...})`. Two problems. First, on failure you learn that **one** of three calls errored, not which. Second and more seriously, per S1 the realistic failure here is not an error at all — `split_max_dli` currently returns a stump, and this block passes. A smoke test that passes when the feature under test is entirely non-functional is worse than no test.

Split them, and assert on the artefact rather than on the absence of an exception:

```
for (sp in c("FIT", "DLi", "DGi")) {
  tr <- doTrees(dat, model, covs, influence = "mat",
               splitter = sp, control = ctl)
  expect_s3_class(tr, "igsca_tree")
  expect_gt(partykit::width(partykit::node_party(tr)), 1L) # it actually split
  expect_identical(attr(tr, "igsca_info")$n_fail_full, 0L)
}
```

`expect_gt(width, 1)` is the assertion that would have caught S1.

4.4 T4. The partition-family tests cannot finish in a test suite

Every test uses the default `igsca_tree_control()`: `R_test = 500L`, `max_cuts = 20L`, `maxdepth = 3L`. The partition family needs a fresh two-group MGA refit per candidate **and** per permutation replicate. Measured on this fixture:

```
one 2-group MGA fit ..... 0.585 s
one permutation_pvalue at R_test = 500 ..... ~4.9 min
one node, 3 covariates, 20 cuts + 500 perms ..... ~15.2 min
one maxdepth = 3 tree (up to 7 inner nodes) ..... ~1.8 h
```

The three partition blocks grow eight such trees between them, so `test-postestimate_doTrees.R` is on the order of **tens of hours**. Define a test control at the top of the file and use it everywhere:

```
ctl <- igsca_tree_control(R_test = 49L, max_cuts = 4L, maxdepth = 2L,
                          minbucket = 100L)
```

`R_test = 49L` keeps the smallest attainable p-value at $1/50 = 0.02 < \alpha$, so splitting is still possible — worth stating explicitly, because anything below `R_test = 19L` makes $p < 0.05$ unreachable and silently guarantees a stump. If the full-fidelity run matters, keep it as a separate `skip_on_ci()` block.

4.5 T5. Nothing asserts the diagnostics the port exists to collect

No test touches `n_fail_full`, `n_fail_node`, `n_fail_resample`, or `root_criteria` — the entire `new_collector()` apparatus is untested, including the node-local scan cache in `argmax_split()`, which is the one piece of genuinely subtle logic in the port (`identical()` on closures as a cache key). Worth one targeted test that a matched selector/splitter pair costs no extra fits while a mismatched pair rescans, e.g. by comparing `n_fail_resample` or by counting calls with a `local_mocked_bindings()` stub on `partition_stat()`.

5 Packaging and API

5.1 P1. NAMESPACE is stale — most of the new API is unreachable

R/helper_doTrees.R carries 25 @export tags. NAMESPACE contains three of them (bdiagFit, candidate_partitions, root_criteria) plus doTrees and the two S3 methods. roxygen has not been re-run since the file was written.

The concrete consequence is user-facing: igsca_tree_control is **not** exported, yet it is doTrees()’s documented control constructor. doTrees(...) works with the default (the default expression is evaluated in the package environment), but

```
cSEM::doTrees(dat, model, covs, control = cSEM::igsca_tree_control(maxdepth = 2L))
#> Error: 'igsca_tree_control' is not an exported object from 'namespace:cSEM'
```

Same for split_max_fitdiff / split_max_dli / split_max_dgi, which the current API requires callers to pass by value. The tests only pass because testthat evaluates inside the package namespace — they cannot catch this class of bug at all.

Run devtools::document(). Then prune: partition_stat, scan_covariate, permutation_pvalue, partition_select, node_group_data, ndt_pools, new_collector, argmax_split, grow_extree, candidate_partitions, and the select_* functions are internal machinery and should lose @export. Adopting B2’s string-valued splitter also removes the need to export the split_max_* kernels. A defensible public surface is doTrees, igsca_tree_control, bdiagFit, root_criteria, and the two S3 methods.

5.2 P2. ::: and :: into the package’s own namespace

Location: R/helper_doTrees.R:572 and R/helper_doTrees.R:608

```
Sc = cSEM::bdiagFit(single_fit, .n_blocks = 2L, .type_vcv = "construct"),
...
cSEM::calculateFIT(mga$fit) - cSEM::calculateFIT(model$object)
```

Both are defined in this package (bdiagFit at R/helper_doTrees.R:26, calculateFIT at R/helper_assess.R:2013). R CMD check flags the ::: form (“a package almost never needs to use ::: for its own objects”), and both break under load_all() if the installed and sourced versions diverge. Drop the prefixes. R/postestimate_doTrees.R:51 already gets this right, calling calculateGSCAErrors() bare where the template used cSEM:::.

5.3 P3. Roxygen blocks are placeholders

Nearly every new block is #' Title with empty @param, @returns, and @examples tags. Empty @param tags produce Undocumented arguments in documentation object warnings, and a bare @examples with no body produces an empty \examples{} section. This is a R CMD check WARNING wall the moment document() is run. Either fill them in or mark the internal ones @keywords internal / @noRd — which, combined with P1’s pruning, removes most of the work.

Two specific errors in the one hand-written block:

- R/postestimate_doTrees.R:6 — @return claims class modelparty and party. Actual class is c("igsca_tree", "constparty", "party") on both paths. There is no modelparty anywhere.
- R/postestimate_doTrees.R:4 — @inheritParams csem_arguments cannot resolve data, model, covariates, influence, splitter, or control. zz_arguments.R:336 documents .covariates (dot-prefixed), and csem() takes .data / .model. None of the six parameters will inherit.

5.4 P4. Argument naming departs from the package convention

Every other exported cSEM entry point takes dot-prefixed arguments

(.data, .model, .approach_weights, ...). doTrees() takes bare data, model, covariates, influence, splitter, control. That is also why P3's @inheritParams silently fails. Renaming to .data / .model / .covariates would fix the docs and the inconsistency at once; the branch is young enough that this is cheap now and expensive later.

5.5 P5. fit_csem() hard-codes the estimator with no override

Location: R/helper_doTrees.R:152--164

```
fit_csem <- function(.data, .model, .id = NULL) {  
  csem(.data = .data, .model = .model, .id = .id,  
    .approach_weights = "GSCA", .disattenuate = TRUE,  
    .conv_criterion = "sum_diff_absolute", .iter_max = 100,  
    .GSCA_modes = "CCMP", .tolerance = 0.0001)  
}
```

Reasonable as a study default, but doTrees() gives the user no way past it — no ..., no argument. Since the tree machinery itself is estimator-agnostic (nothing outside partition_stat()'s FIT branch is GSCA-specific), threading ... from doTrees() through try_fit() into csem() costs almost nothing and makes doTrees() usable for PLS-PM trees too. At minimum, document that the estimator is fixed.

5.6 P6. No input validation

doTrees() validates influence (via match.arg) and nothing else. Worth adding, since all three produce errors from deep inside partykit or csem():

```
stopifnot(  
  "`data` must be a data.frame" = is.data.frame(data),  
  "`covariates` must be columns of `data`" =  
    length(covariates) > 0L && all(covariates %in% names(data))  
)  
missing_ind <- setdiff(indicators, names(data))  
if (length(missing_ind)) {  
  stop2("The following indicators are not columns of `data`: ",  
    paste(missing_ind, collapse = ", "))  
}
```

6 What the port got right

Worth recording, since these are deliberate improvements over the template rather than accidents:

- **Indicators from the model, not the data.** parseModel(model)\$indicators (R/postestimate_doTrees.R:22) replaces the template's setdiff(names(data), covariates). The template silently treated any stray column as an indicator; the port does not, and now tolerates extra columns in data. The two orderings differ (x31, x32, ... , x11 vs x11, x12, ...), which is why the recorded snapshot's model formula reads ~x31 + x32 + x33 + x11 + ... — expected, not a bug.
- **The unified dispatch is the right cut.** Branching on influence rather than exposing two constructors correctly reflects that the COIN and partition families differ only in how the node statistic is produced.
- **influence_ssr → influence_vec** reads better next to influence_mat. The stale "COIN_ssr" references in the comments (R/helper_doTrees.R:113, :514) should follow the rename.

- The scan-cache, minbucket-filtering, and `.Machine$double.eps` flooring logic all survived the port intact and behave correctly under test.

7 Suggested order of work

#	Item	Why first
1	B1 — port <code>idx_permutation()</code> + <code>ndt_dists()</code>	Nothing else in the partition family is testable until this lands
2	B2 — make <code>splitter</code> a <code>match.arg()</code> string	Changes the call signature; do it before writing assertions against it
3	S2 — restore <code>cc\$bonferroni <- isTRUE(control\$bonferroni)</code>	One line, silently wrong today
4	T4 — test-scoped control constant	Without it you cannot iterate on the tests at all
5	T1–T3, T5 — regenerate snapshots, replace <code>length()==5</code> and <code>expect_no_error()</code>	Turns the suite into something that can detect S1
6	S1 — count and surface dead-kernel scans	Needs the tests from 5 to verify
7	S3 — leaf refits, then fix <code>coef()</code> / <code>plot()</code>	Largest behavioural change; independent of the rest
8	P1–P6 — <code>document()</code> , prune exports, drop <code>:::</code> , validate inputs	Cleanup; do once the API in B2/P4 has settled